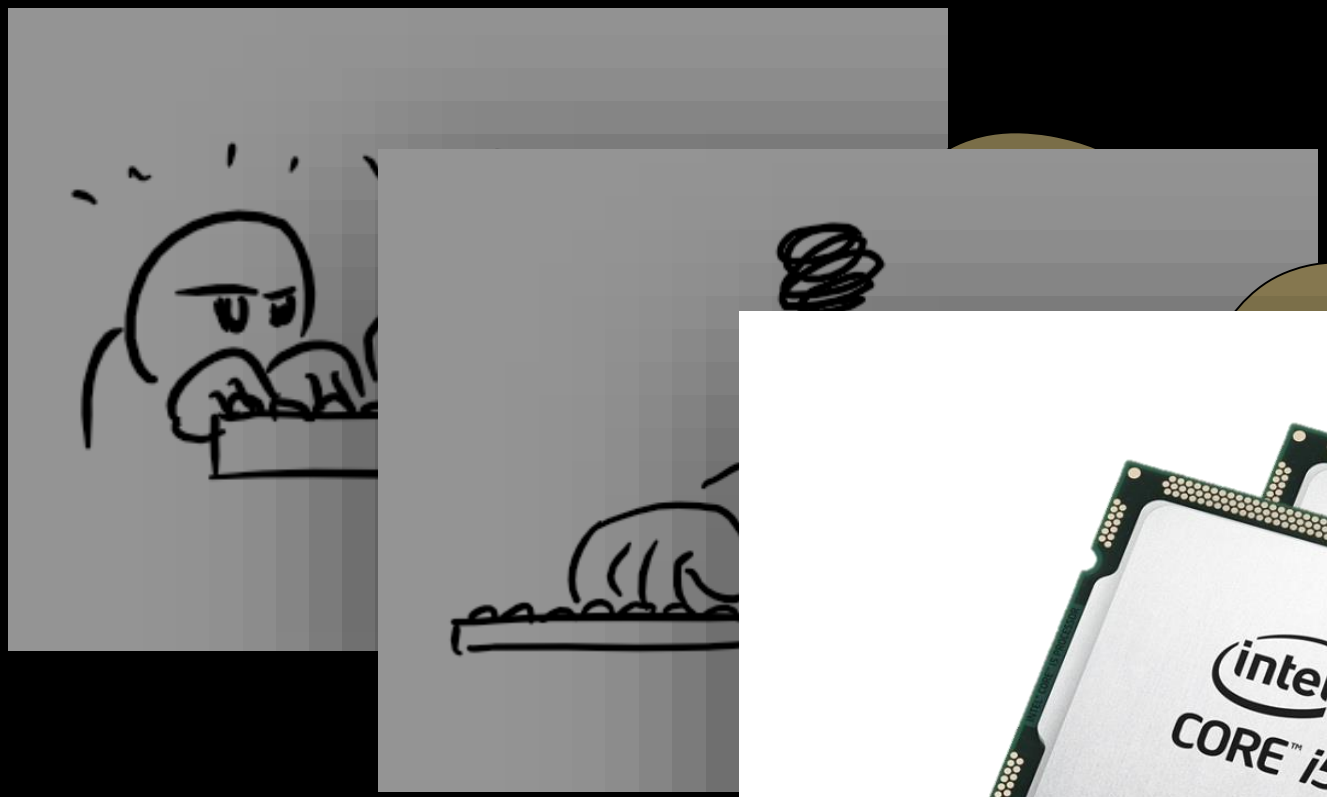
Bedeutung von Paralleler Programmierung

Michael Faes

Doktorand

ETH zürich

Was ist parallele Programmierung?



Entwickeln von Anwendungen,
die mehrere CPUs verwenden

Wo gibt's mehrere CPUs?

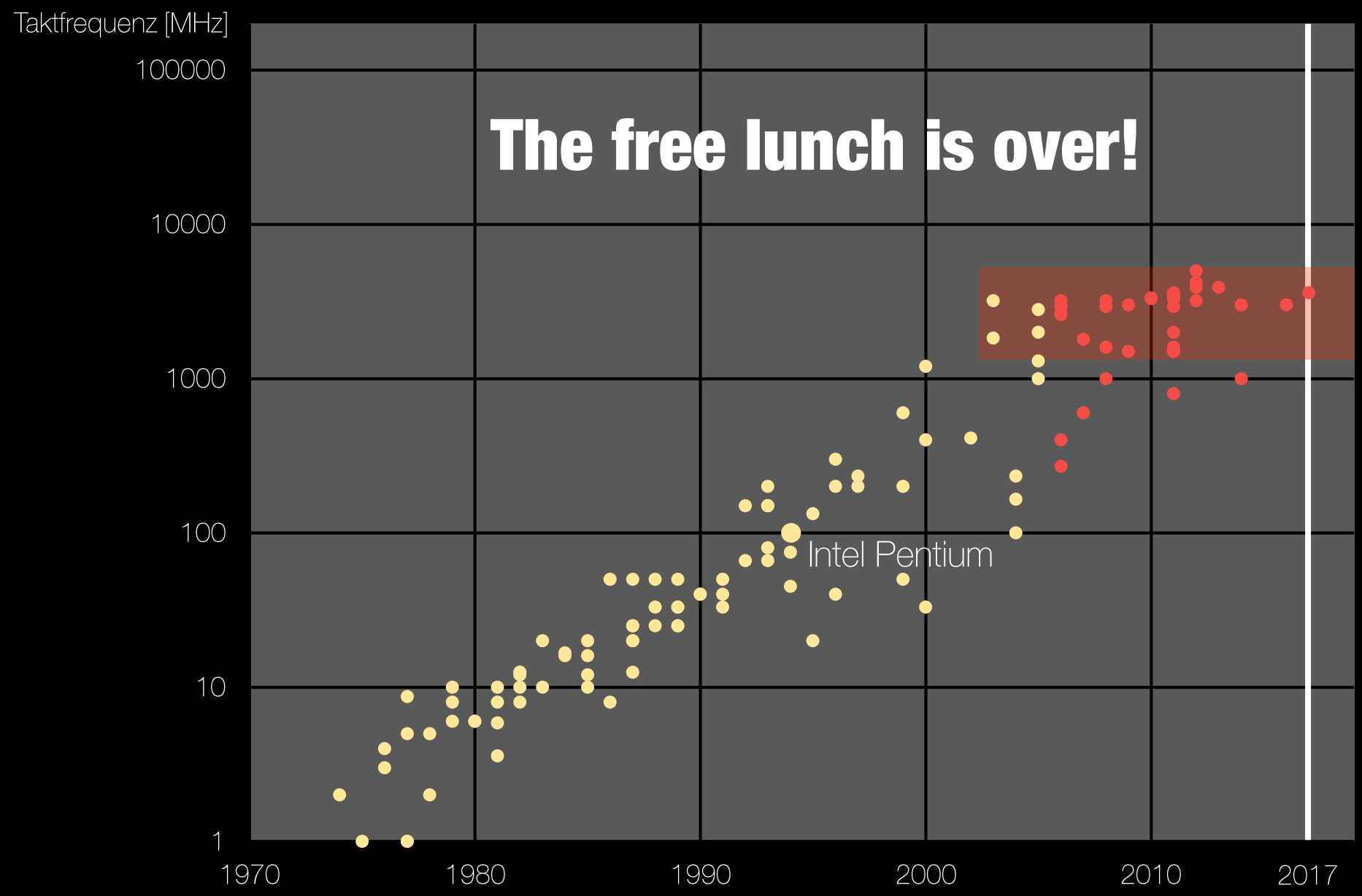
336 cores

8 cores

10 cores

4 cores

Warum parallel programmieren?



Daten: https://en.wikipedia.org/w/index.php?title=Instructions_per_second&oldid=783515052

Was ist Parallelismus?

La Ratatouille niçoise

Für 6 – 8 Personen

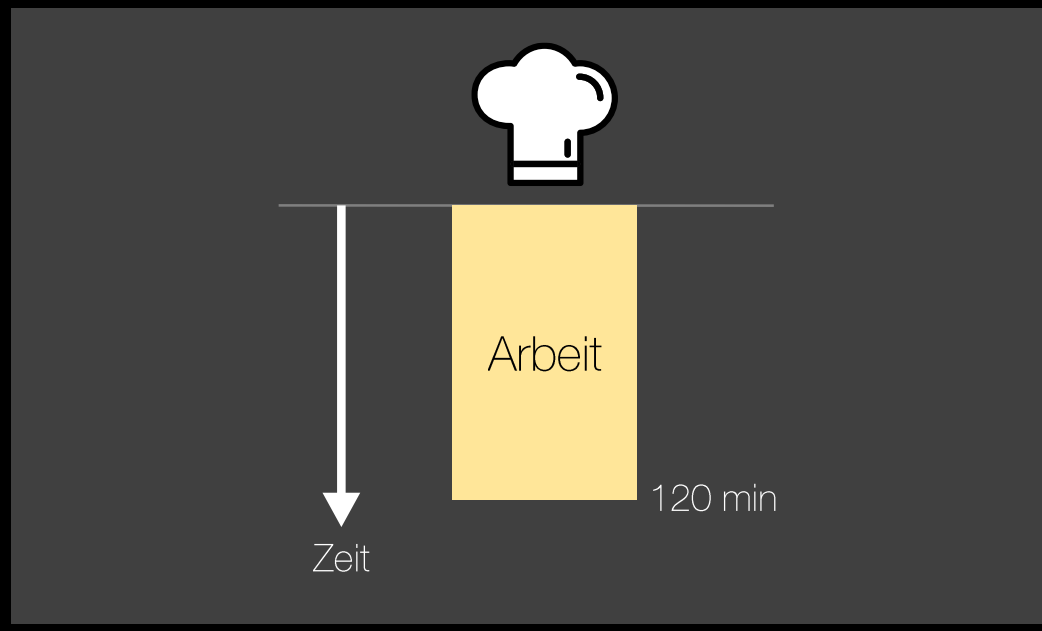
1. *Die Tomaten kurz in kochendes Wasser tauchen, herausheben, abschrecken, häuten, entkernen und in Würfel schneiden.*

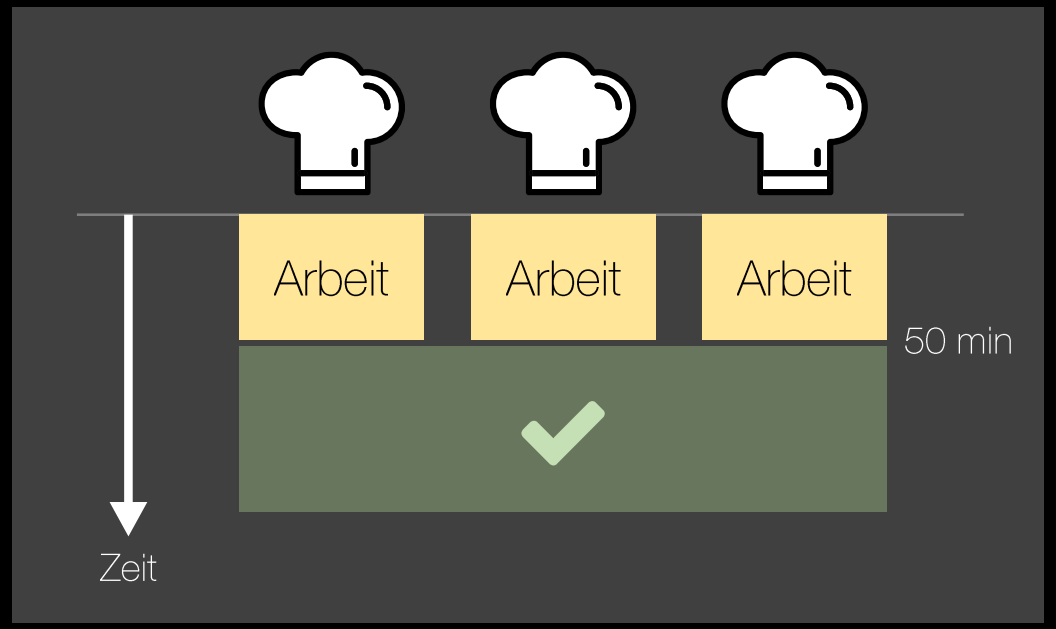
2. *Die Zwiebeln schälen, halbieren und in Streifen schneiden. Die Knoblauchzehen schälen und in feine Scheibchen schneiden. Thymianblättchen von den Zweigen zupfen.*

3. *Olivenöl erhitzen, Zwiebeln und Knoblauch andünsten. Tomaten und Thymian beifügen, würzen. Dann 30 Minuten schmoren.*

4. *Inzwischen die Peperoni halbieren, entkernen und in Streifen schneiden. Olivenöl erhitzen, Peperonistreifen beigeben, würzen und knapp weich dünsten.*

5. *Die Auberginen ungeschält in fingerdicke*

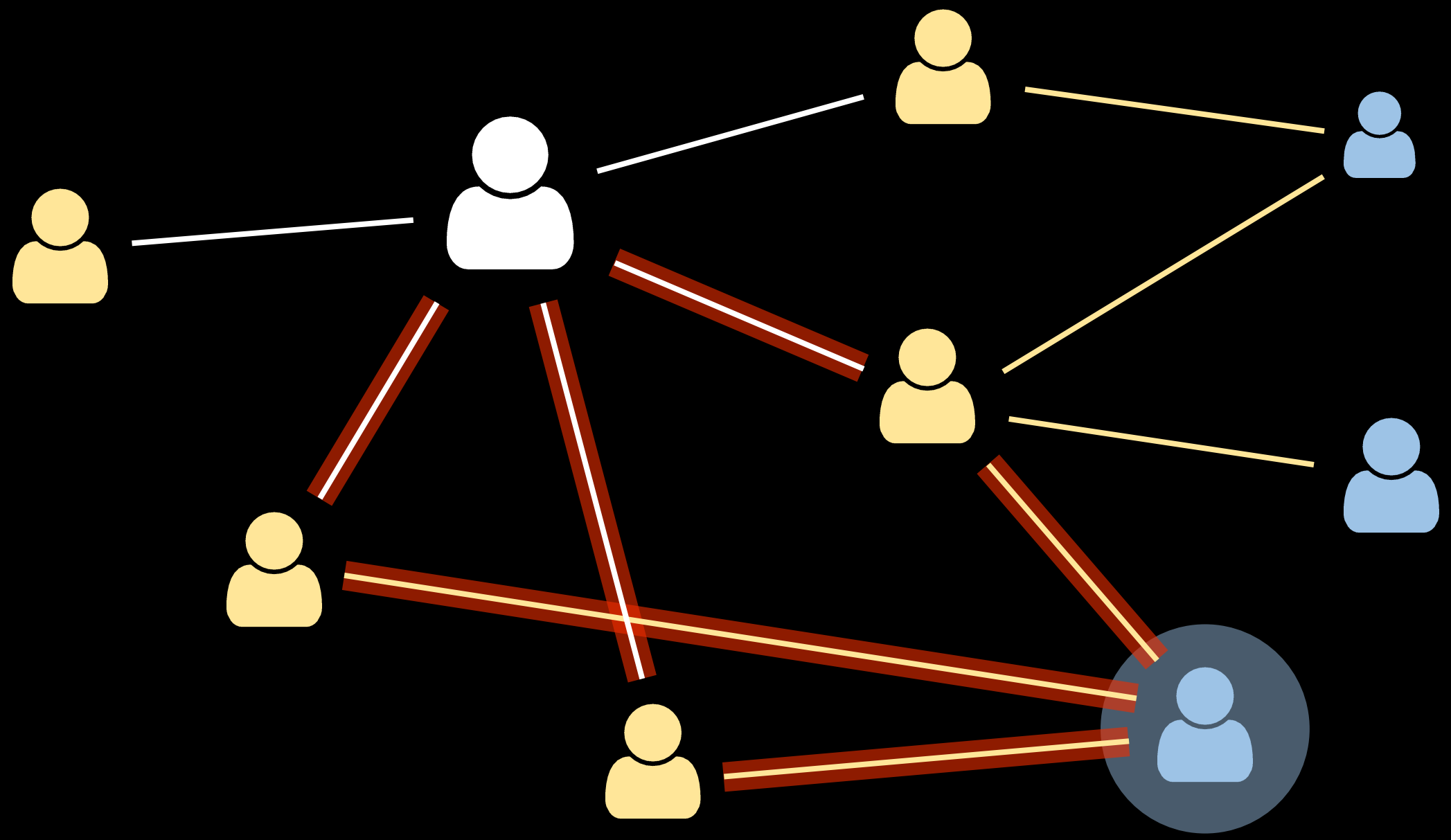




Parallelisierung einer Applikation

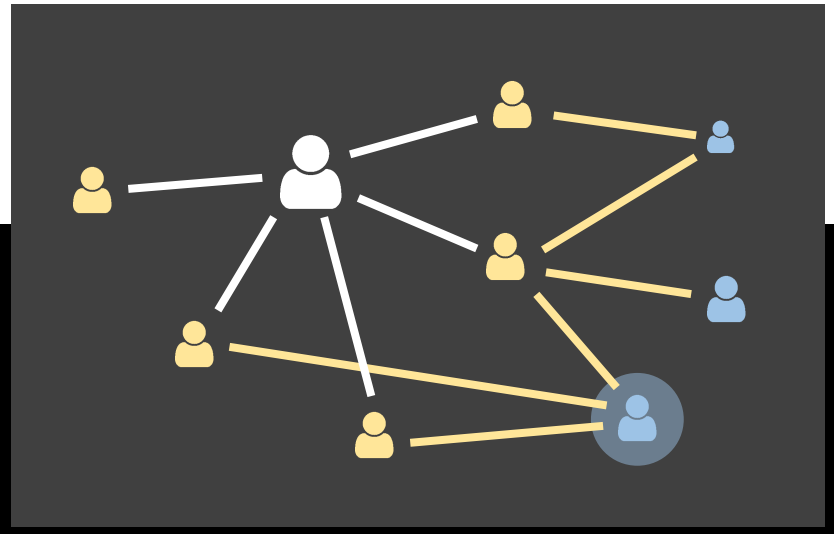
(in Java)

Beispiel: Soziales Netzwerk



Freunde vorschlagen

```
1   public List<User> friendSuggestions(User user) {
2       List<User> result = new ArrayList<>();
3       for(User friend : user.friends) {
4           for(User fiendsFriend : friend.friends) {
5               int commonFriends = countCommonFriends(user, friendsFriend);
6               if(commonFriends >= 3)
7                   if(!result.contains(friendsFriend))
8                       result.add(friendsFriend);
9           }
10      }
11      return result;
12  }
```

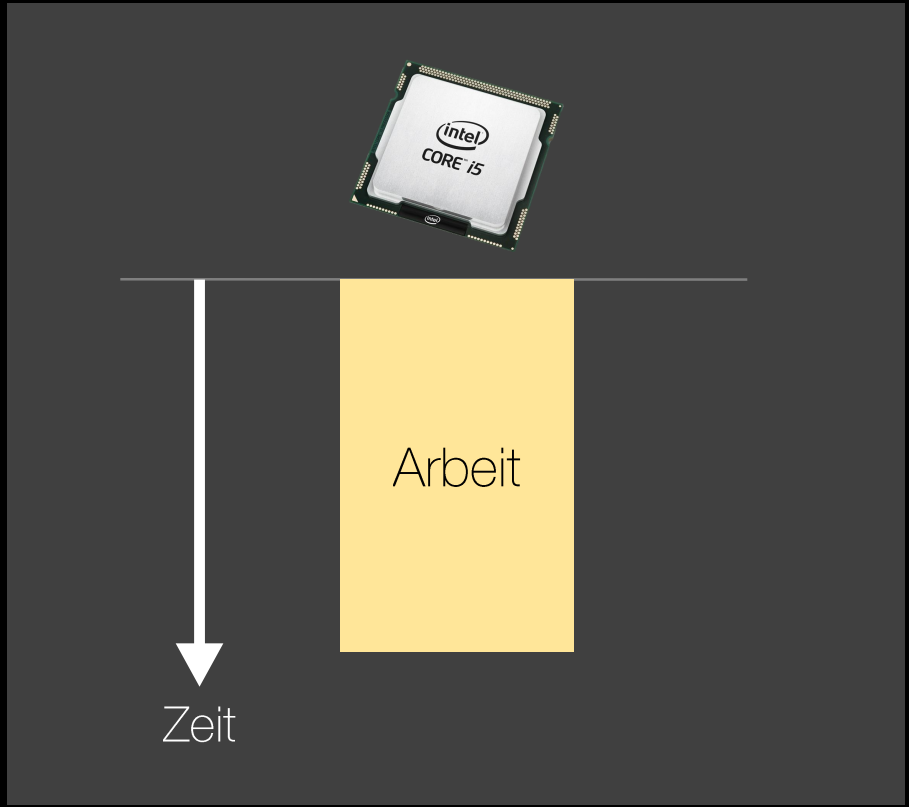


Freunde vorschlagen, parallel!

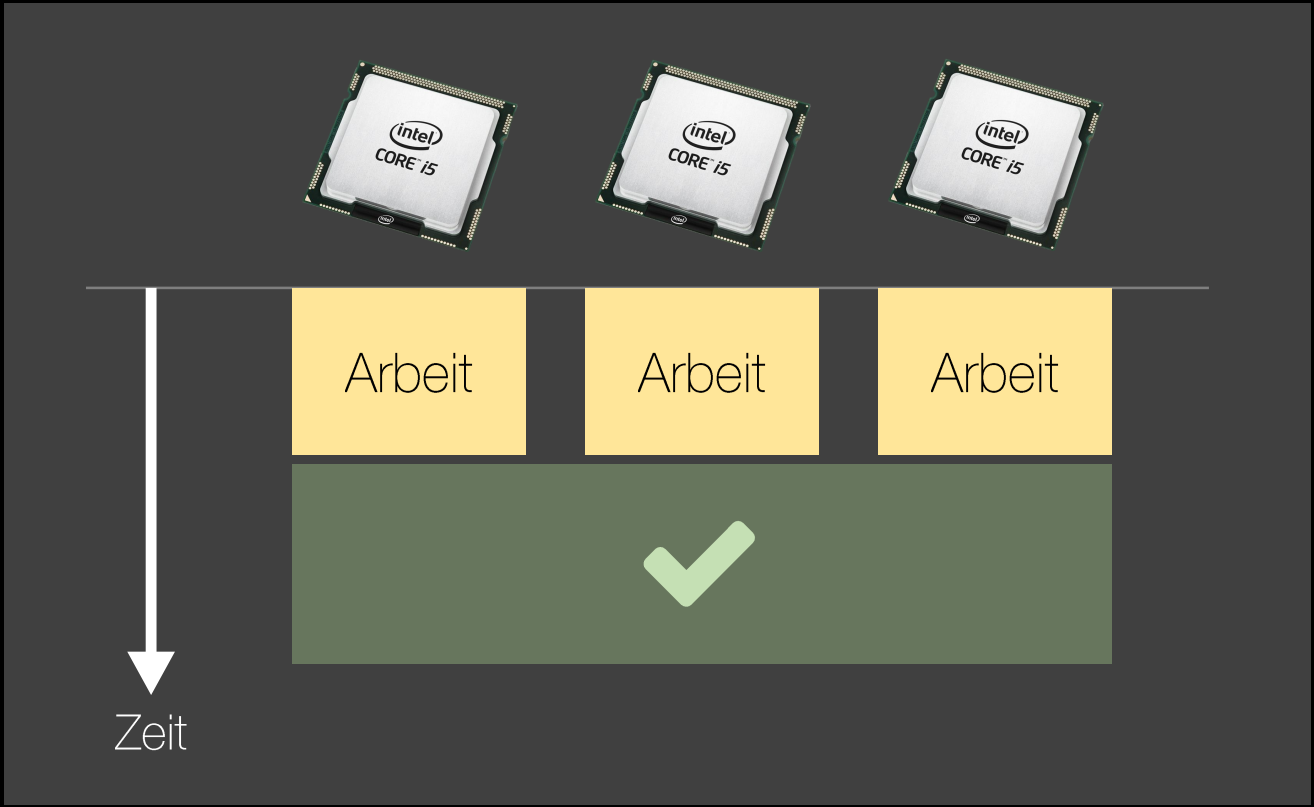
```
1  public List<User> friendSuggestions(User user) {
2      List<User> result = new ArrayList<>();
3      (...)
4      for(User friend : user.friends) {
5          Thread t = new Thread(() -> {
6              for(User fiendsFriend : friend.friends) {
7                  int commonFriends = countCommonFriends(user, friendsFriend);
8                  if(commonFriends >= 3)
9                      if(!result.contains(friendsFriend))
10                         result.add(friendsFriend);
11             }
12         });
13         t.start();
14         (...)
15     }
16     (...)
17     return result;
18 }
```

Resultat

Sequenziell



Parallel



Problem: PP ist schwierig!

Paralleles `friendSuggestions()` war falsch!

	Resultat
Sequenziell:	[Anna, Marc, Rolf, Sophie]
Parallel:	[Anna, Marc, **Rolf**, **Rolf**, Sophie]

Wie ist das möglich?

```
9   if(!result.contains(friendsFriend))
10     result.add(friendsFriend);
```

Fehler in `friendSuggestions()`

Resultat

```
!result.contains(…)
result.add(…)
```

```
!result.contains(…)
result.add(…)
```

[ ]
[Anna]
[Anna]
[Anna]

```
!result.contains(…)
result.add(…)
```

```
!result.contains(…)
result.add(…)
```

[Anna, Marc]
[Anna, Marc]
[Anna, Marc, Rolf]
[Anna, Marc, **Rolf**, **Rolf**]

Zeit

Ungewollte Verzahnungen

Wenn mehrere Threads auf das selbe Objekt zugreifen, müssen sie das Objekt "sperren" (engl.: locken), um ungewollte Verzahnungen zu verhindern.

```
9   synchronized(result) {
10      if(!result.contains(friendsFriend))
11          result.add(friendsFriend);
12  }
```

🔒 sperre **result**

führe Operationen auf **result** durch

🔓 entsperre **result**

`friendSuggestions()` korrigiert

Resultat

```
synchronized(result){
!result.contains(…)

result.add(…)
}
```

```
synchronized(result){

!result.contains(…)
result.add(…)
}
```

[Anna, Marc]

[Anna, Marc]

[Anna, Marc, Rolf]

[Anna, Marc, Rolf]

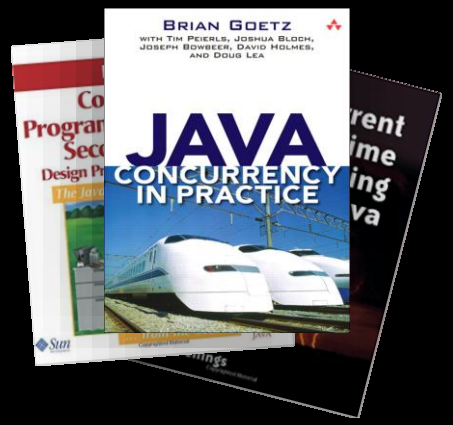
[Anna, Marc, Rolf]

Zeit

Probleme mit paralleler Programmierung

- Paralleles Programm hat richtig ausgesehen
- Paralleles Programm hat manchmal funktioniert!
- Nur eine spezifische Verzahnung macht Probleme
- Für richtige Programme gibt es Milliarden von möglichen Verzahnungen

In Wirklichkeit gibt es noch viel mehr Probleme…



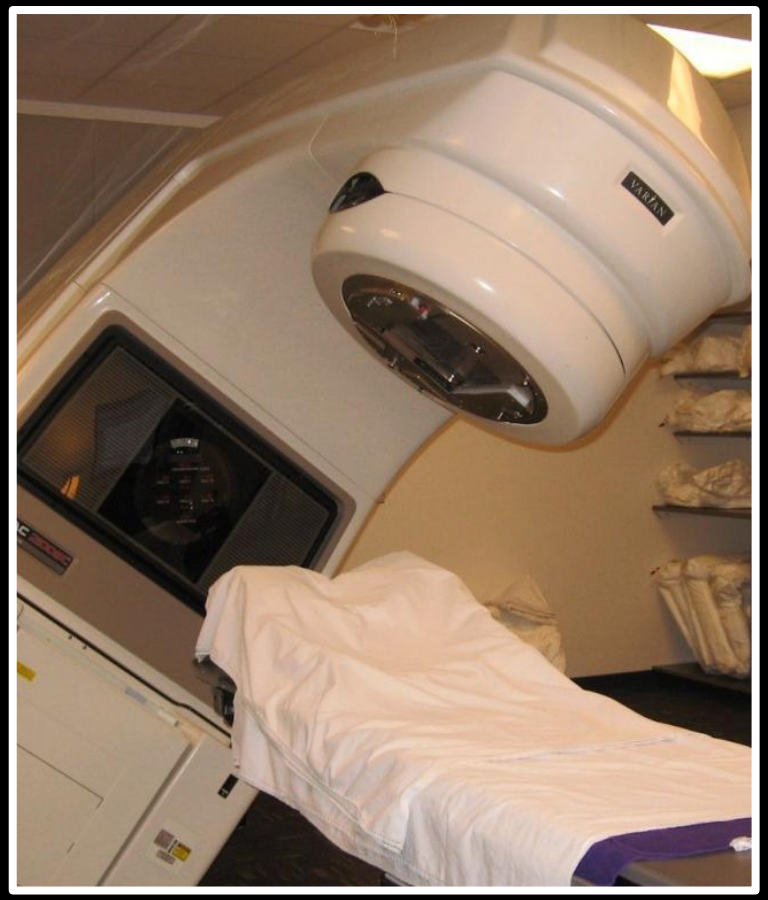
Reale Probleme?

2003: Stromausfall USA

Race condition verhinderte, dass ausgefallene Hochspannungsleitungen Alarm auslösten

Kein Strom für über 50 Millionen Menschen!

1982: Therac-25



Linearbeschleuniger für Strahlentherapie

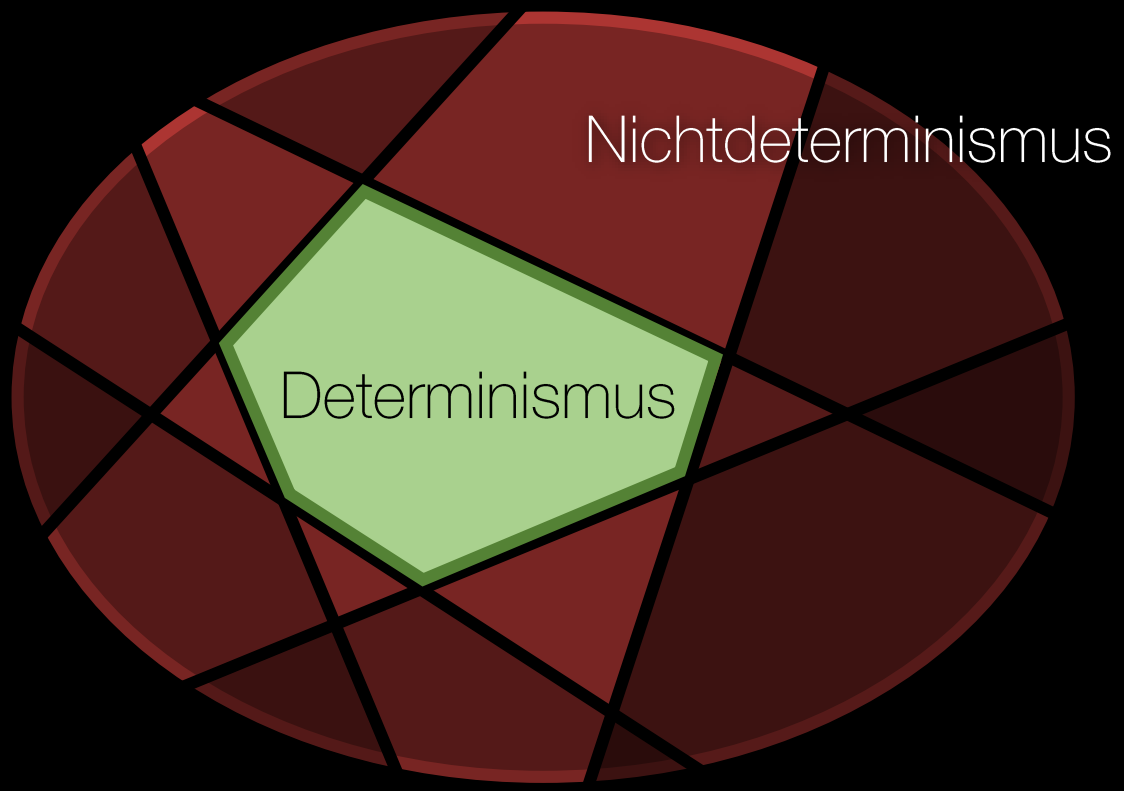
Race condition führte zu 100x höherer Strahlung und kostete 3 Menschen das Leben!

Eigentliches Hauptproblem: Nichtdeterminismus

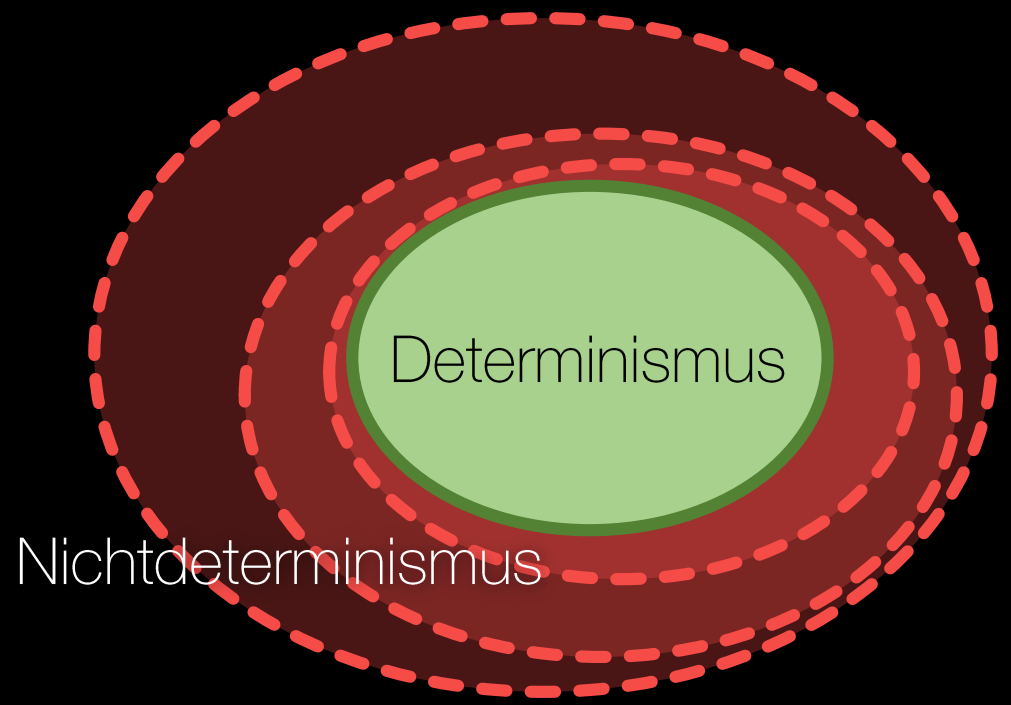
> *Ein Programm kann mehrere verschiedene Berechnungen durchführen, ohne dass ihm vorgegeben wird, welche.*

Deterministische Parallele Programmierung

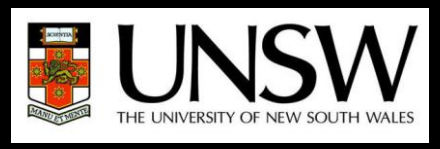
Herkömmliche PP



Deterministische PP



Deterministische PP ist Forschungsthema

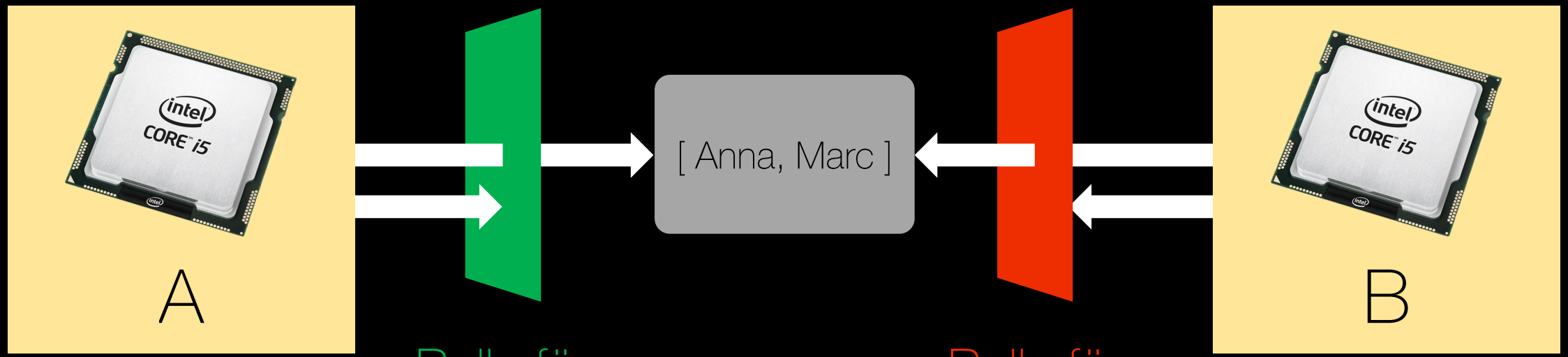


ETH zürich

Meine Doktorarbeit: Deterministische PP mit “Rollen”

- Java-ähnliche, objekt-orientierte Programmiersprache
- Wird zu Java-Sourcecode übersetzt
- Garantiert deterministische Ausführung

Parallel Roles – Hauptidee

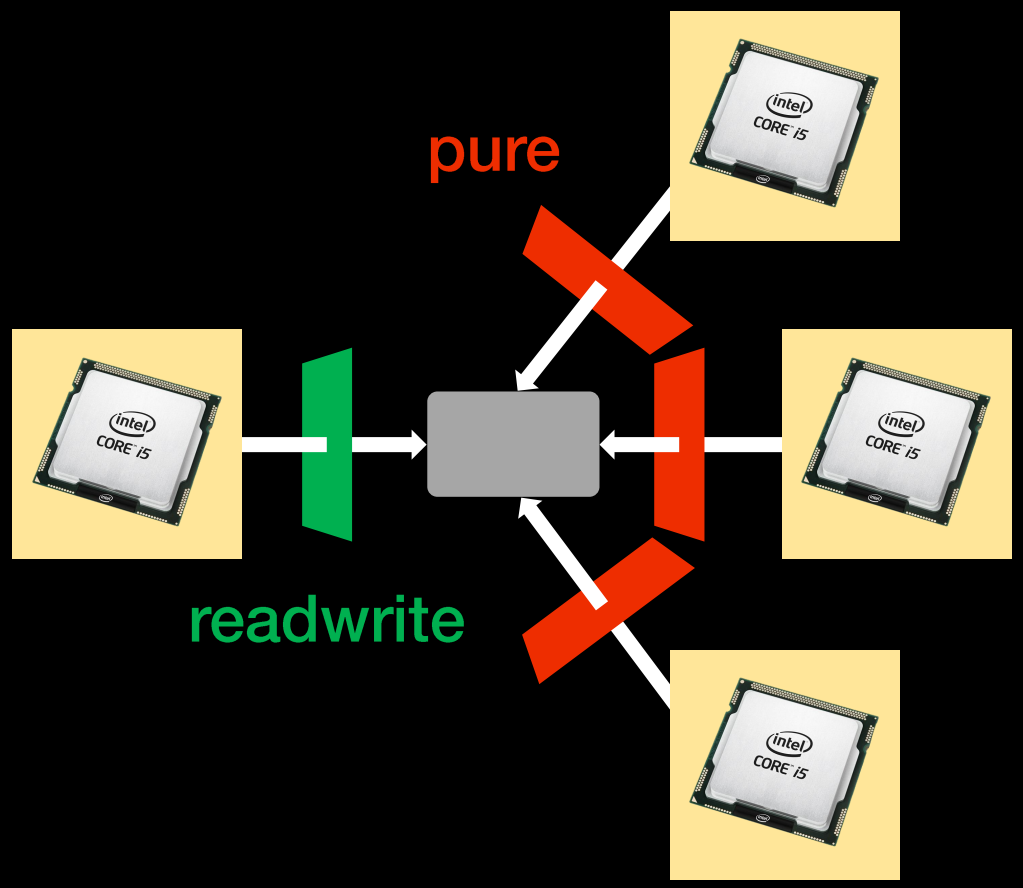


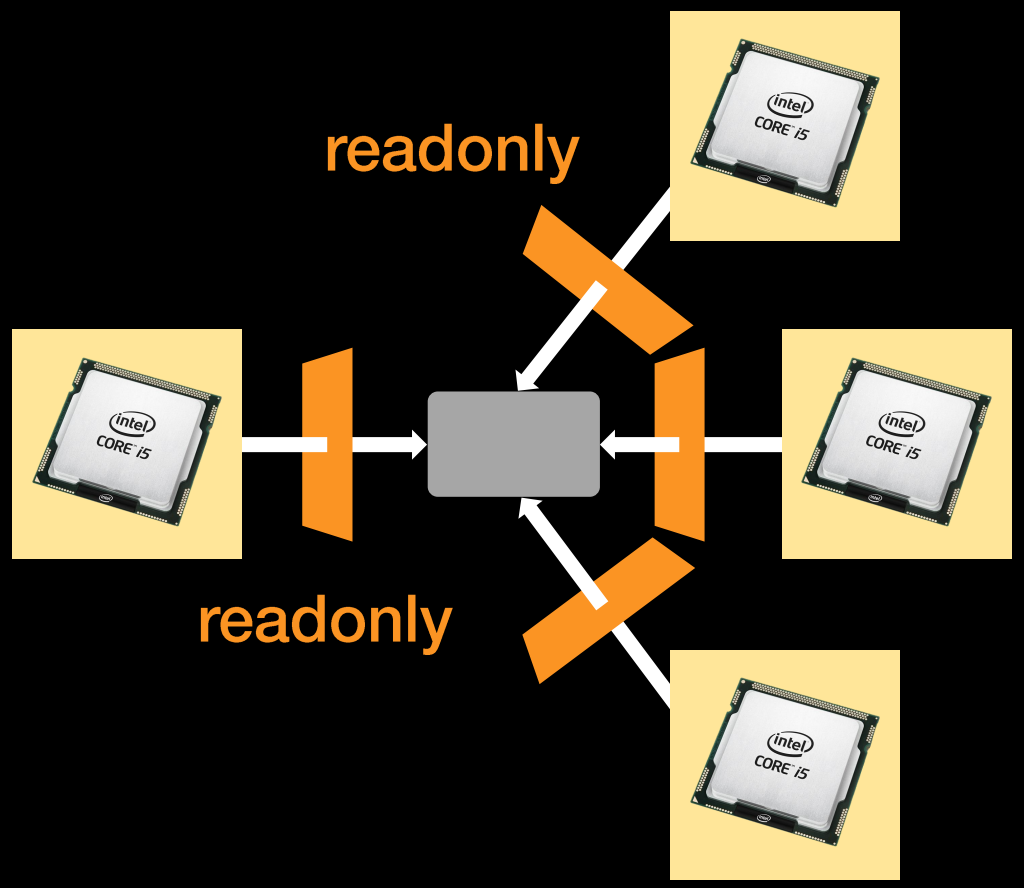
Rolle für Thread A

Rolle für Thread B

	readwrite	readonly	pure
schreiben	✓	✗	✗
lesen	✓	✓	✗

Rollen führen zu deterministischer PP

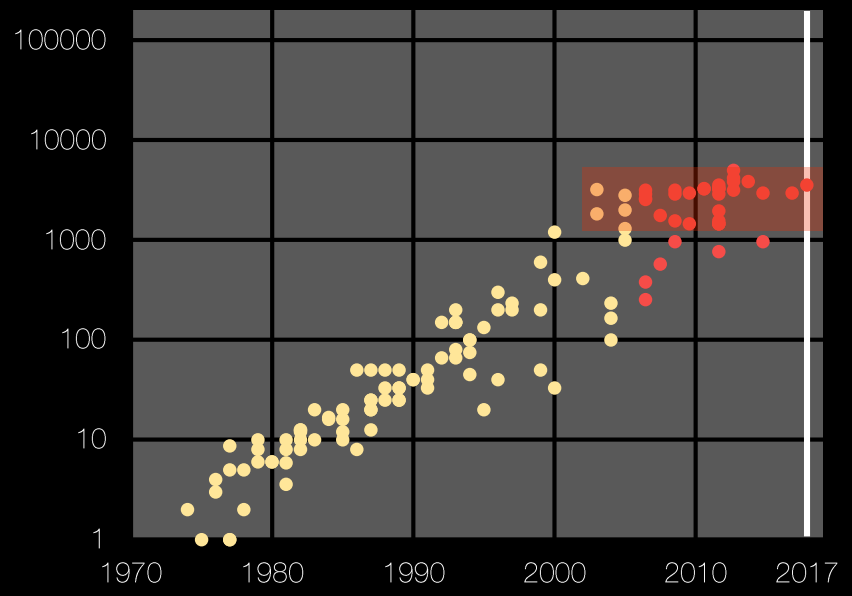




Resultat: Programm sieht sequenziell
aus und verhält sich auch so!

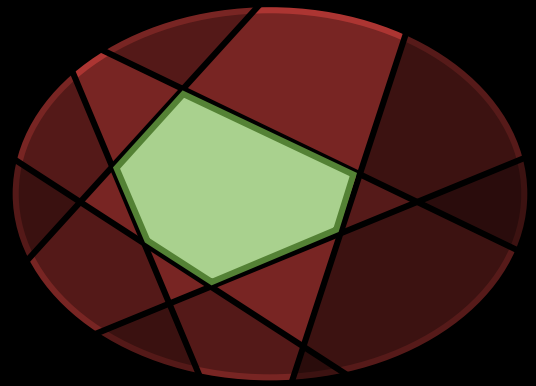
Parallel Roles Beispiel

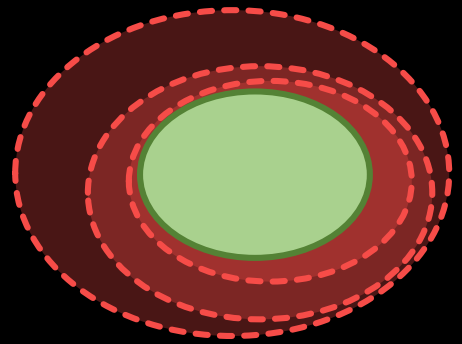
```
1   class IdeaEncryption {
2      val plaintext: readwrite Array[byte]
3      val encrypted: readwrite Array[byte]
4      ...
5
6      def readwrite parallelEncrypt(tasks: int): {
7         val slices = encrypted.partition(..., tasks);
8         for(var i = 0; i < tasks; i++)
9            start encrypt(plaintext, slices.get(i));
10     }
11
12     task pure encrypt(src: readonly Array[byte],
13                       dst: readwrite Slice[byte]): {
14        ...
15     }
16  }
```



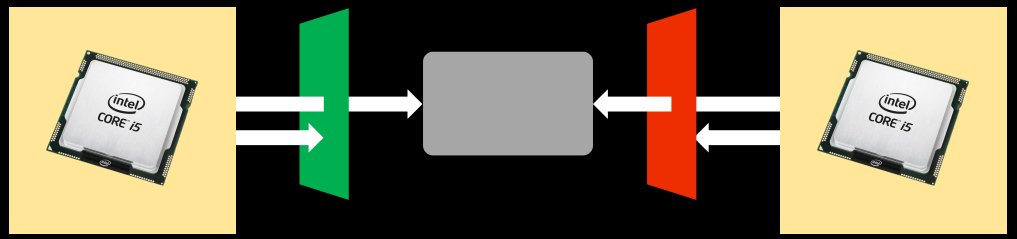
PP ist nötig.

PP ist schwierig.





Deterministische PP hilft.



Parallel Roles ist neuer Ansatz für DPP.

Anhang

Freunde vorschlagen, parallel! (komplett)

```
1   public List<User> friendSuggestions(User user) {
2       List<User> result = new ArrayList<>();
3       List<Thread> threads = new ArrayList<>();
4       for(User friend : user.friends) {
5           Thread t = new Thread(() -> {
6               for(User fiendsFriend : friend.friends) {
7                   int commonFriends = countCommonFriends(user, friendsFriend);
8                   if(commonFriends >= 3)
9                       if(!result.contains(friendsFriend))
10                          result.add(friendsFriend);
11              }
12          });
13          t.start();
14          threads.add(t);
15      }
16      for(Thread t : threads) t.join();
17      return result;
18  }
```